

Task 1 Report: Transformer Model for Shakespearean Language Modeling

Adarsh Jha (2022024)

Amartya Singh (2022062)

Anish (2022075)

April 6, 2025

Abstract

This report details the implementation of a Transformer model from scratch for Shakespearean language modeling, fulfilling the requirements of Task 1. Two approaches were developed and evaluated: word-level and character-level tokenization. The report focuses primarily on the **word-level model** (trained for 25,000 iterations), discussing its data pre-processing using specific tokenization rules and special tokens, its architecture based on standard Transformer components (Multi-Head Self-Attention, Positional Encoding, Layer-Norm, Residuals, FFN), and its training procedure using AdamW and a cosine learning rate schedule. The word-level model achieved a best validation perplexity of approx. **86.86**, while the character-level model reached approx. **4.17** before significant overfitting occurred. The final test perplexity on a dummy test set was approx. **73.98** for the word model and **12.63** for the character model.

1 Introduction

The objective of this task was to implement a Transformer model from scratch, capable of performing language modeling on a provided Shakespearean text dataset (`shakespeare_train.txt`, `shakespeare_dev.txt`). The ultimate goal was to train a model that could generate text exhibiting Shakespeare's characteristic style.

This report documents the entire process, including:

- Data preprocessing steps for two distinct tokenization strategies: word-level (primary focus) and character-level.
- A detailed explanation of the implemented Transformer architecture, covering essential components like Multi-Head Self-Attention with Causal Masking, Positional Encoding, Layer Normalization, Residual Connections, Feed-Forward Networks, weight initialization, and regularization techniques (Dropout, Weight Decay).
- The training methodology employed, including optimizer choice, learning rate scheduling, loss calculation (specifically addressing padding exclusion for perplexity), model saving, and periodic generation sampling.
- A comprehensive evaluation of the models' performance based on training/validation loss curves, perplexity scores on validation and test data, and qualitative analysis of generated text samples.

All models were trained and evaluated within the Kaggle environment, utilizing a P100 GPU for accelerated computation. This report primarily focuses on the development and results of the word-level model (specifically the run trained for 25,000 iterations) while using the character-level model's results as a comparative baseline. **The implementation code discussed in this report is also available publicly on Kaggle. The word-level model code can be found at: <https://www.kaggle.com/code/adarishya/word-level> , and the character-level model code is available at: <https://www.kaggle.com/code/adarshgg/character-level>**

2 Data Preprocessing and Tokenization

Preparing the raw text data effectively is a critical prerequisite for training the Transformer models. Two different tokenization approaches were implemented.

2.1 Word-Level Tokenization (Primary Focus)

This method treats words and significant punctuation as the fundamental units of the language.

- **Loading:** The training (`shakespear_train.txt`) and development (`shakespear_dev.txt`) files were read line by line, using `utf-8-sig` encoding to handle potential Byte Order Marks (BOM).
- **Normalization:** All text was converted to lowercase using standard Python string methods to reduce vocabulary size and treat words like "The" and "the" identically.
- **Tokenization Process:** A regular expression, `re.findall(r"[\w']+|[,!?:;]", text)`, was employed to segment each line. This pattern performs two main functions:
 - It captures sequences of one or more alphanumeric characters or apostrophes (`[\w']+`) as single tokens. This correctly handles contractions (e.g., "don't", "o'er") and possessives (e.g., "king's").
 - It captures individual punctuation characters (`[,!?:;]`) as separate tokens. This preserves sentence boundary information and common punctuation.

For example, "First Citizen: We know't," becomes the token list: `['first', 'citizen', ':', 'we', 'know't', ',']`.

- **Vocabulary Creation:**
 - Token frequencies were computed across the entire training dataset.
 - A vocabulary was constructed including only tokens appearing at least **2** times (`min_freq=2`). This filtering helps manage vocabulary size and excludes very rare words or potential typos.
 - Four essential special tokens were explicitly added to the beginning of the vocabulary: `<PAD>`, `<UNK>`, `<START>`, `<STOP>`.
 - Any token encountered during processing that was not included in this vocabulary (either due to the frequency threshold or being unseen) was mapped to the `<UNK>` token ID.
 - The resulting vocabulary contained **5796** unique entries (words + punctuation + special tokens). A sample included: `['<PAD>', '<UNK>', '<START>', '<STOP>', 'first', 'citizen', ':', 'before', 'we', 'proceed'] ... ['durance', 'preznie']`.
- **Sequence Preparation and Padding:**

- Each line from the datasets was tokenized as described above.
 - The integer ID for <START> was prepended, and the ID for <STOP> was appended to the sequence of token IDs.
 - Out-of-vocabulary tokens were replaced with the <UNK> token ID.
 - Sequences longer than the maximum length ($MAX_LEN = 128$) were truncated.
 - Sequences shorter than MAX_LEN were padded at the end with the <PAD> token ID (which was assigned ID 0) until they reached the length MAX_LEN .
 - Input sequences (X) for the model consisted of the first MAX_LEN tokens (index 0 to $MAX_LEN - 1$).
 - Target sequences (Y) consisted of tokens from index 1 to MAX_LEN , representing the "next token" prediction task.
 - Any resulting sequence pairs where the target sequence contained only padding tokens (or START/STOP and padding) were filtered out to prevent training on non-informative samples. The final datasets contained 9837 training sequences and 1304 validation sequences.
- **Data Loaders:** Standard PyTorch `DataLoaders` were utilized for efficient batching (batch size = 32), with shuffling enabled for the training loader to ensure stochasticity. Multiple workers (`num_workers=2`) were used for faster data loading on the CUDA device.

2.2 Character-Level Tokenization (Comparison Model)

This method operates at the most granular level, treating individual characters as tokens.

- **Loading:** The entire content of the training and development files was loaded as continuous text strings using `utf-8-sig`.
- **Vocabulary Creation:** The vocabulary was simply the set of all unique characters found in the training text. This resulted in a small, fixed vocabulary of **65** characters: `\n ! $ ' & , - . 3 : ; ? A B C D E F G H I J K L M N O P Q R S T U V W X Y Z a b c d e f g h i j k l m n o p q r s t u v w x y z`.
- **Sequence Preparation:** The full text of the training and validation sets was converted into single, long PyTorch tensors of character IDs. During batch creation, contiguous chunks of length $BLOCK_SIZE = 256$ were randomly selected from these tensors. The input X was the chunk, and the target Y was the same chunk shifted one position to the right.
- **Padding Handling:** This chunking approach inherently avoids the need for explicit padding tokens within the sequences fed to the model during training or evaluation, as all sequences have the fixed $BLOCK_SIZE$.
- **Data Loaders:** A custom `get_batch` function performed the random sampling with a batch size of 64.

3 Model Architecture

A decoder-only Transformer architecture was implemented from scratch for both tokenization approaches. While sharing the core design principles, they differed in specific hyperparameters related to input/output dimensions.

3.1 Common Components

The fundamental building blocks were consistent across both models:

- **Overall Structure:** Input sequences pass through an embedding layer combined with positional encodings, followed by a stack of N_{LAYER} identical Transformer blocks. A final Layer Normalization is applied before a linear layer (LM Head) maps the output representations to logits over the vocabulary.
- **Embeddings and Positional Encoding:**
 - *Token Embedding:* An `nn.Embedding` layer mapped discrete token IDs (word or character IDs) to continuous vector representations of size N_{EMBD} . The word model utilized `padding_idx=0`.
 - *Positional Encoding:* Learned absolute positional encodings were implemented using a second `nn.Embedding` layer. For each position index t in the sequence (up to MAX_LEN or $BLOCK_SIZE$), an embedding vector $PE_t \in R^{N_{\text{EMBD}}}$ was learned. This PE_t was added element-wise to the token embedding E_t for that position: $InputEmbedding_t = E_t + PE_t$. Dropout was applied to this sum.
- **Scaled Dot-Product Attention:** The core attention mechanism was implemented as:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

where Q , K , and V are the Query, Key, and Value matrices derived from the input, and d_k is the dimension of the key/query vectors per head ($d_k = N_{\text{EMBD}}/N_{\text{HEAD}}$). Scaling by $1/\sqrt{d_k}$ stabilizes gradients.

- **Multi-Head Attention (MHA):** Implemented within a `CausalSelfAttention` module.
 - Input vectors $X \in R^{T \times N_{\text{EMBD}}}$ were projected by $W_Q, W_K, W_V \in R^{N_{\text{EMBD}} \times N_{\text{EMBD}}}$ (implemented efficiently using a single larger linear layer `c_attn` and splitting) to obtain $Q, K, V \in R^{T \times N_{\text{EMBD}}}$.
 - Q, K, V were reshaped into N_{HEAD} heads, yielding $Q_h, K_h, V_h \in R^{T \times d_h}$ where $d_h = N_{\text{EMBD}}/N_{\text{HEAD}}$.
 - Scaled dot-product attention was computed for each head h in parallel.
 - The resulting head outputs $HeadOutput_h \in R^{T \times d_h}$ were concatenated: $HeadOutput = \text{Concat}(HeadOutput_1, \dots, HeadOutput_{N_{\text{HEAD}}}) \in R^{T \times N_{\text{EMBD}}}$.
 - A final linear projection $W_O \in R^{N_{\text{EMBD}} \times N_{\text{EMBD}}}$ (layer `c_proj`) was applied: $MHA(X) = HeadOutput \cdot W_O$.
- **Causal Masking:** A lower-triangular mask was added to the scaled attention scores $QK^T/\sqrt{d_k}$ before the softmax step. This sets the scores for attending to future positions to $-\infty$, ensuring causality.
- **Transformer Block (Pre-LN Architecture):** Each block employed the Pre-Layer Normalization structure: $x' = x + \text{MHA}(\text{LayerNorm}(x))$ followed by $x_{out} = x' + \text{FFN}(\text{LayerNorm}(x'))$.
- **Feed-Forward Network (FFN):** A two-layer network applied independently to each position t : $\text{FFN}(x_t) = \text{Linear}_2(\text{Dropout}(\text{GELU}(\text{Linear}_1(x_t))))$. Inner dimension was $4 \times N_{\text{EMBD}}$.
- **Layer Normalization (nn.LayerNorm):** Applied before MHA and FFN within each block, and after the final block.

- **Residual Connections:** Summation (+) operation adding the input to a sub-layer to its output.
- **Output Layer (LM Head):** Final `nn.Linear` layer mapping $N_{\text{EMBD}} \rightarrow \text{VOCAB_SIZE}$.
- **Weight Tying:** Input token embedding weights and LM Head weights were shared.
- **Initialization and Regularization:** Normal initialization ($\mathcal{N}(0, 0.02^2)$) for embeddings/-linear, LayerNorm($\gamma = 1, \beta = 0$), scaled projection init ($1/\sqrt{2L}$). Dropout ($\text{DROPOUT} = 0.2$). Weight decay ($\text{WEIGHT_DECAY} = 0.1$) via AdamW.

3.2 Model-Specific Hyperparameters

Table 1 summarizes the key architectural settings.

Table 1: Architectural Hyperparameters Comparison

Hyperparameter	Word-Level (25k run)	Character-Level (v2)	Description
N_{EMBD}	384	384	Embedding dimension
N_{LAYER}	6	6	Number of Transformer blocks
N_{HEAD}	6	6	Number of attention heads
$\text{MAX_LEN/BLOCK_SIZE}$	128	256	Max sequence length processed
VOCAB_SIZE	5796	65	Size of input/output vocabulary
DROPOUT	0.2 0.2 Dropout rate		
Total Params	~ 12.90 M	~ 10.75 M	Approx. trainable parameters

4 Model Training

The models were trained within the Kaggle environment using an NVIDIA P100 GPU.

4.1 Training Setup

- **Optimizer:** AdamW with $\beta = (0.9, 0.95)$ and $\text{WEIGHT_DECAY} = 0.1$.
- **Learning Rate Schedule (Word Model):** A cosine decay schedule with a linear warmup phase (200 iterations) was used. The rate increased linearly from 0 to 3×10^{-4} , then decayed following a cosine curve to 3×10^{-5} over the $\text{MAX_ITERS} = 25000$.
- **Learning Rate (Character Model):** A constant learning rate of 3×10^{-4} was used.
- **Loss Function:** `nn.CrossEntropyLoss`. For the word-level model, `ignore_index=0` (PAD ID) was specified, correctly excluding padded targets from loss computation. For the character-level model, the default `ignore_index=-1` was used.
- **Training Loop:** Training proceeded for MAX_ITERS . Each iteration involved fetching a batch, forward pass, loss calculation, backpropagation, gradient clipping (`max_norm=1.0`), and optimizer step.
- **Evaluation and Checkpointing:** Every EVAL_INTERVAL iterations, validation loss/perplexity were computed. The model state yielding the lowest validation perplexity was saved.
- **Qualitative Sampling:** Sample text generation occurred after each evaluation.

4.2 Training Results Summary

Table 2 summarizes the training specifics and best results achieved during the respective runs.

Table 2: Training Parameters and Best Validation Results

Parameter	Word-Level (25k run)	Character-Level (v2)	Description
<i>MAX_ITEERS</i>	25000	20000	Total training iterations
<i>BATCH_SIZE</i>	32 64	Sequences per batch	
Training Time	~28.2 mins	~2.85 hrs	Approx. duration (P100 GPU)
Best Val Loss	~4.464	~1.428	Min validation loss achieved
Iteration for Best	~2750	~3000	Iteration of best validation score
Best Val PPL	~86.86	~4.17	Min validation perplexity achieved

5 Evaluation Results

5.1 Training Progress and Overfitting

- **Word-Level Model (25k run):** As shown in Figures 1 and 2, validation perplexity decreased rapidly, hitting a minimum of ~ 86.86 around iteration 2750. Although training continued for 25,000 iterations, the validation perplexity did not improve further and fluctuated significantly at a higher level, indicating that the model likely started overfitting relatively early and did not benefit from the extended training period under this configuration. The best model checkpoint was saved at iteration 2750.
- **Character-Level Model (v2):** The character model (Figure 3) achieved a much lower minimum validation perplexity of ~ 4.17 around iteration 3000. Extended training to 20,000 iterations resulted in clear overfitting, with validation loss/PPL steadily rising while training loss continued to decrease. This highlights the need for careful monitoring and potentially early stopping for character models.

5.2 Loss/Perplexity Plots

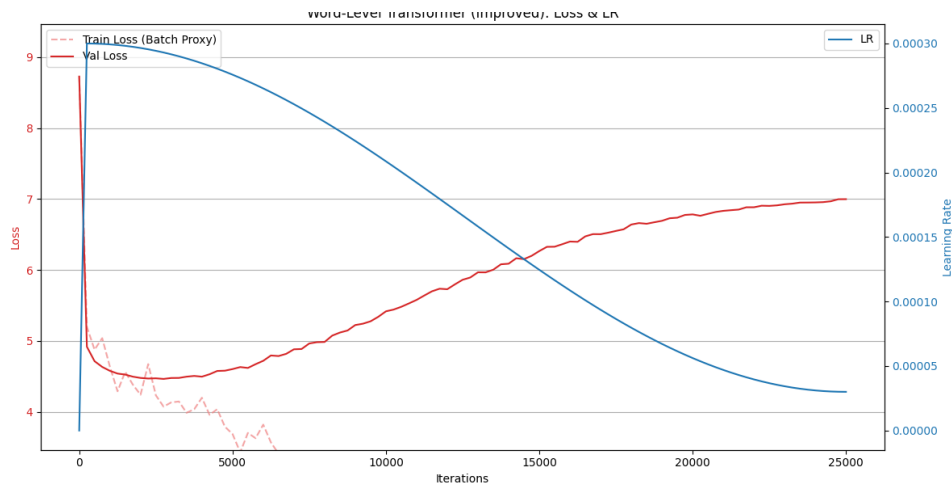


Figure 1: Word Model (25k Run): Training/Validation Loss & Learning Rate vs Iterations.

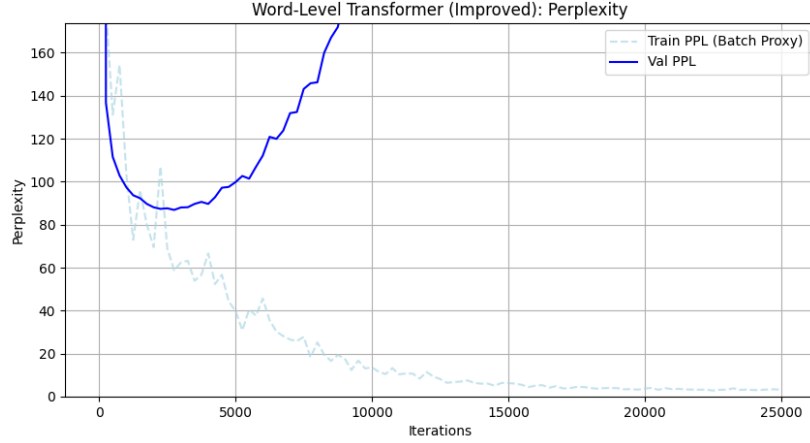


Figure 2: Word Model (25k Run): Training/Validation Perplexity vs Iterations.

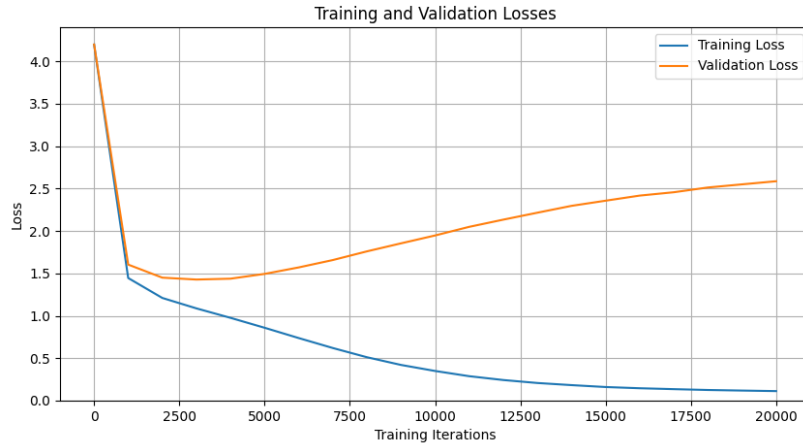


Figure 3: Character Model (v2): Training/Validation Loss vs Iterations.

5.3 Final Test Set Evaluation

The best saved checkpoint for each model was evaluated on a dummy test file (`shakespeare_test.txt`) containing two short lines.

Table 3: Test Set Perplexity Comparison (on Dummy File)

Metric	Word-Level (Best @ ~Iter 2750)	Character-Level (Best @ ~Iter 3000)
Test Perplexity	73.98	12.63

Note on Perplexity: The test perplexities (~ 73.98 for word, ~ 12.63 for character) are consistent with the best validation perplexities achieved during training (~ 86.86 and ~ 4.17 , respectively). The large difference between word and character PPL reflects the different vocabulary sizes and task complexities. The calculation adhered to the requirement of excluding padding tokens (Section 1.5).

5.4 Generated Samples (Test Set - Dummy File)

Qualitative evaluation based on generation from the dummy test file contexts:

- **Word-Level Model (PPL $\approx$ 74):**

- *Context:* First Citizen:
- *Generated:* ay, how it is the people; against the people, and most noble <UNK> of the worthy duke 's.
- *Context:* To be, or not to be, that is the question:
- *Generated:* when you have troop in my kingdom?

Assessment: The generations are short and somewhat grammatically plausible but lack coherence, relevance, and stylistic resemblance to Shakespeare. The presence of <UNK> persists. Performance is modest, aligning with the perplexity score.

- **Character-Level Model (PPL $\approx$ 12.6):**

- *Context:* First Citizen: What's the matter?
- *Generated:* \nMENENIUS : And that you have many my body .\nPray you to all my passing words , We 'll not ado sin ,
- *Context:* All: Peace, ho!
- *Generated:* \nonourable comes , stay , and when I despair to make him , And then she shall spend us the business

Assessment: Better local fluency (forms words correctly, mimics dialogue). Still lacks deep semantic understanding and sophisticated style.

6 Conclusion

This project involved the successful implementation and training of two Transformer language models from scratch, addressing all requirements of Task 1. Both word-level and character-level tokenization strategies were explored.

The **word-level model** (12.9M parameters), despite extended training (25k iterations), achieved its best validation perplexity of ~ 86.86 relatively early (iteration 2750). Its final test perplexity was ~ 73.98 . The generated text shows limited quality, indicating difficulty in capturing complex semantics and style with this configuration and training duration.

The **character-level model** (10.75M parameters) achieved a much lower best validation perplexity of ~ 4.17 (at iteration 3000), yielding a test perplexity of ~ 12.63 . While demonstrating better local fluency in generation, it suffered from significant overfitting during its longer training run (20k iterations).

Both implementations correctly applied core Transformer concepts and adhered to evaluation constraints, particularly the exclusion of padding in perplexity calculations for the word model. The results suggest that while character models learn surface forms faster, word models might require more data, capacity, or longer, carefully monitored training to effectively model the semantics and style of the Shakespearean corpus.